THERMOBENCH / REV B

Build it. Measure it. Explain it.

Complete Windows setup, breadboard assembly, live dashboard, electrical checks, fault tests and interview preparation.

Prepared for Aaron Trujillo | 19 October 2026

Start with software demo mode. Then verify the actual board and parts before wiring. Each numbered step gives a concrete action and a check before moving on.

Includes new desktop dashboard and automated CSV analysis. The original engineering report remains the reference for the schematic, calculations, specifications and synthetic SPC illustrations.

Evidence status: nine Python tests passed. GUI source syntax checked. Actual GUI operation, ESP32 firmware compilation, USB acquisition, physical circuit and thermal accuracy require local verification.

Prepared for Aaron Trujillo. AI-assisted design and code.

1. What is ready

The package includes firmware, a desktop dashboard, a serial logger, automated CSV checks, synthetic SPC examples, source tests and the original 18-page engineering report. Nine Python tests passed in the authoring environment. Dashboard syntax was checked, but its window and USB operation were not tested here. Firmware remains uncompiled and no hardware measurements were taken here.

The dashboard labels every session HARDWARE or SYNTHETIC. It records voltage, temperature, state and device timestamps. The analyzer evaluates CSV validity, timestamp order, mean reporting rate, continuous valid temperature, fixed-input repeatability, and optional mean voltage versus a DMM. A passing synthetic run verifies software behavior, not the physical prototype.

2. Identify parts first

• Read the ESP32 module marking and board name. The supplied circuit assumes an ORIGINAL ESP32-WROOM DevKit with accessible GPIO34. An ESP32-C3, S2 or S3 requires a verified ADC pin and possibly other changes; do not copy GPIO34 blindly.

• Find the thermistor bag, order or datasheet. The supplied temperature model assumes 10k ohms at 25 C and beta 3950 K. Room-temperature resistance alone cannot establish beta.

• Gather breadboard, jumpers, USB DATA cable, DMM, 10k resistor, 1k resistor, 100nF capacitor, thermistor and two other fixed resistors such as 6.8k and 3.3k.

• A reference thermometer with stated accuracy/uncertainty is required for temperature-accuracy claims. You can complete electrical tests and a response demo without one.

• Power off and remove each resistor from the circuit before measuring its resistance. Record actual resistance. An ordinary DMM reading does not make a resistor a calibrated standard.

Checkpoint: exact board and NTC identification written down. If beta is unknown, continue electrical tests but label all temperature results MODEL ESTIMATE.

3. Extract and install

Extract the ZIP using Windows Extract All, not by opening files inside the compressed folder. Put it in C:\Users\YOUR_WINDOWS_NAME\Documents\ThermoBench. Open the extracted project folder in VS Code. Use Terminal > New Terminal and choose PowerShell. The terminal folder must contain bench.py, dashboard.py and station.py.

Run these commands one at a time:

```
py --version
py -m venv .venv
.\.venv\Scripts\python -m pip install -r requirements.txt
.\.venv\Scripts\python -m tkinter
.\.venv\Scripts\python -m unittest -v
```

The Tkinter command should open a small test window; close it. Expect 9 passing tests. If 'py' is not recognized, install Python from python.org with the launcher and Tcl/Tk support, then reopen VS Code. No virtual-environment activation or PowerShell policy changes are required.

Optional: save actual installed versions with:

```
.\.venv\Scripts\python -m pip freeze > installed_versions.txt
```

4. Practice in demo mode

Run:

```
.\.venv\Scripts\python dashboard.py
```

Click Demo. Confirm SYNTHETIC is visible, the temperature is near 25 C, and the trend moves. Click Start CSV recording; leave it for at least 65 seconds; click Stop recording, then Disconnect. Your CSV is under captures with a synthetic_ prefix. Copy its exact filename into the command below, replacing EXAMPLE.csv:

```
.\.venv\Scripts\python station.py captures\EXAMPLE.csv --out analysis_demo
```

Open analysis_demo\summary.json and capture.png. All steady-run checks should pass in a normally scheduled demo. Investigate timing failures rather than editing results. The synthetic banner must remain visible in demonstrations and reports. Save this as a fallback software demo, not as hardware evidence.

5. Assemble the low-voltage circuit

Unplug USB before wiring. Inspect breadboard row connections and split power rails.

Connections:

• ESP32 3V3 -> breadboard positive rail.

• ESP32 GND -> breadboard ground rail.

• R1, 10k -> between positive rail and a free row named TP1.

• NTC -> between TP1 and ground.

• R2, 1k -> between TP1 and ESP32 GPIO34.

• C1, 100nF -> between GPIO34 and ground.

Do not connect the ADC to VIN/5V. The 1k resistor does not make it a 5V-tolerant input. Check the original report page 4 for the vector schematic. Keep wires short. Position the NTC away from the ESP32's warmth.

Start with a measured 10k fixed resistor in place of the thermistor. This creates a stable electrical test before thermal testing. For matched 10k resistors, TP1 should be approximately half the actual 3V3 rail.

Checkpoint: wiring checked with USB unplugged; no unintended power-to-ground connection. Power on only after this check.

6. Set measured constants

With USB connected and DMM set to DC voltage, measure 3V3 to GND. Measure TP1 and the ADC node to GND. TP1 and the ADC node should be nearly equal at steady state. Record all values. Do not use resistance mode on the powered circuit.

Open thermobench.ino. Update:

```
constexpr float VS_MV=3200.0f, RF=10000.0f,
                R25=10000.0f, BETA=3950.0f;
```

VS_MV is your measured rail in millivolts. RF is your measured R1 in ohms. R25 and BETA come from the identified sensor datasheet or a defensible calibration. Do not enter the thermistor's unknown-room-temperature resistance as its 25 C resistance.

bench.py uses separate nominal model constants. Its demo remains synthetic with Vs=3.2 V; it does not calibrate the device. station.py analyzes reported measurements and does not silently change firmware calibration.

7. Install ESP32 support and upload

Install Arduino IDE from arduino.cc. Follow Espressif's official installation guide. In IDE Preferences, add this board-manager URL if needed:

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

Open Boards Manager, install esp32 by Espressif Systems, then select the exact board and USB port. Copy thermobench.ino into a folder named thermobench and open it from there. Click Verify, then Upload. Save the successful build output and core version.

Open Serial Monitor at 115200 baud. Expect a header followed by rows such as:

```
ms,mv,temp_c,status
1000,1600.00,25.000,NORMAL
```

The numeric row is an illustrative format, not your expected measured result. With a known resistor installed, temperature is only its equivalent modeled NTC temperature.

If the board does not appear: try another known data cable; compare Windows Device Manager before/after connecting; identify the USB interface before installing a driver. For upload failures, check board/port selection and the board manufacturer's reset/BOOT procedure. Do not change unrelated settings blindly.

Checkpoint: successful compile/upload and stable, readable serial rows. Close Serial Monitor before opening another program on the same COM port.

8. Run the live dashboard

From the project folder:

```
.\.venv\Scripts\python -m serial.tools.list_ports
.\.venv\Scripts\python dashboard.py
```

Click Refresh, choose your actual COM port, then Connect USB. Confirm HARDWARE is visible. Opening the port may reset the board; wait for fresh rows. Watch for rejected rows. Start recording and leave the fixed resistor input untouched for at least 65 seconds. Stop recording before disconnecting.

Expected: roughly 5 rows per second, stable voltage, no stale-data indicator. Files are saved in captures with a hardware_ prefix. Do not open Serial Monitor at the same time. The dashboard closes recording on a timestamp restart to keep sessions separate.

9. Validate the electrical measurement chain

Use the fixed 10k resistor first. Measure ADC-node voltage with the DMM while the recording runs. Record the reference in mV. For example, only if your meter actually reads 1.600 V, use 1600 below. Replace EXAMPLE.csv and reference with actual values:

```
.\.venv\Scripts\python station.py captures\EXAMPLE.csv --out analysis_10k --dmm-mv 1600 --
tolerance-mv 30
```

The 30 mV limit is a preliminary engineering acceptance choice, not a manufacturer guarantee or a certified uncertainty. The script compares mean ADC mV with your entered DMM value. It also checks timing, parser validity and a <=0.15 C model-equivalent steady-state standard deviation. It does not establish true thermal accuracy.

Repeat after powering off and replacing the lower resistor with measured 6.8k and 3.3k values. Compute expected TP1 voltage from:

```
Vexpected = Vrail * Rlower / (R1 + Rlower)
```

At nominal 3.20 V, R1=10k, example lower resistors give: 10k -> 1.600 V; 6.8k -> 1.295 V; 3.3k -> 0.794 V. Use actual values for your predictions. Compare DMM versus prediction and ADC versus DMM separately. Record PASS/FAIL without hiding failures. The analyzer automates ADC-versus-DMM checks; divider prediction comparison is manual.

Checkpoint: three resistor checks documented with measured R1, lower resistance, supply, predicted voltage, DMM voltage and ADC mean. A discrepancy guides debugging.

10. Demonstrate temperature and faults

Power down, restore the identified NTC, and restart the dashboard. Record 65 seconds after the sensor and environment settle. Start a separate recording for a gentle hand-warming demo. Warm only the sensor, then release it: resistance and voltage should decrease as the modeled temperature rises. Hand warming may never reach the 45 C alarm; that is normal.

Fault tests, one change at a time: 1. Power off, remove the NTC connection to ground, power on: expect HIGH_FAULT and blank temperature. 2. Power off, restore NTC: expect valid readings. 3. Power off, bypass the NTC by connecting TP1 to GND, power on: expect LOW_FAULT and blank temperature. 4. Power off, remove bypass and restore the circuit.

Do not short 3V3 directly to GND. The low-fault test is through the installed R1. These power-cycle tests validate fault states; they do not measure response latency to a live fault. A guarded switch fixture is required for a proper live <=1-second fault-latency test.

Hysteresis test with a potentiometer, only if available:

• Replace NTC with a pot used as a variable resistor from TP1 to GND; tie wiper to the grounded end terminal. Keep R1 installed.

• Adjust until indicated temperature is about 46 C: expect HOT.

• Adjust to about 44 C: expect HOT retained.

• Adjust to about 42 C: expect NORMAL.

• Adjust back to about 44 C: expect NORMAL retained.

These resistance substitutions test the conversion/state logic electrically, not real temperature accuracy. Do not heat the sensor just to force an alarm.

11. Verify temperature accuracy if equipment supports it

Use an identified reference thermometer with stated accuracy/uncertainty. Place both sensing tips close together in a suitable controlled setup and wait for stable readings. Avoid contact with the ESP32 or fingers. Bare non-rated sensors must not be immersed; a liquid bath needs a rated probe and proper isolation from the electronics.

Record reference and DUT at several achievable stable temperatures. The report proposes 15, 25, 35, 45 and 55 C, but do not improvise hazardous heating/cooling to achieve them. Narrow the demonstrated range if your setup cannot support the full range. Hand warming cannot verify those points or +/-1 C accuracy.

For each point: record reference value, DUT average, signed error, spread, setup, stabilization rule and comparison uncertainty. Use separate data to fit calibration and validate it. A proposed guardband is |error| + expanded comparison uncertainty <=1 C; only apply it when that uncertainty is justified. Otherwise report observed error and limitations, without a certified PASS.

12. Real graphs and SPC

station.py creates the actual time/voltage and time/temperature graph for a recording. Analyze an unchanging settled session for repeatability; do not label warm-up/cooldown scatter as stationary noise.

The existing command below regenerates the SYNTHETIC SPC teaching example only:

```
.\.venv\Scripts\python bench.py demo --out results
```

For defensible real SPC, collect independent stabilized check runs at an unchanged reference condition; use one consistent run statistic per run. Adjacent 5 Hz samples may be correlated. Investigate baseline stability before freezing limits. For individual observations, MR_i=abs(x_i-x_(i-1)); sigma_est=mean(MR)/1.128; I limits=mean(x) +/-3*sigma_est; MR upper limit=3.267*mean(MR). Freeze baseline limits for later monitoring. Do not claim Cp/Cpk from a drifting, unvalidated process.

The Rev B analyzer does not automatically qualify independence, compute real SPC limits, or calculate thermal uncertainty. Keep synthetic SPC labeled; real SPC and capability remain pending until a suitable dataset exists.

13. Final documentation and interview demo

Keep a build record with board identity, NTC identity, code changes you made, actual software versions, resistor/supply measurements and setup photos. Preserve failed and passing CSVs with notes. Avoid screenshots containing personal account details.

Bring the original report plus a one-page measured-results summary with:

• Project purpose and architecture.

• One photo of your actual build and the schematic.

• Three electrical test results and actual fault states.

• A real stability graph and clearly stated limitations.

• Your contributions, AI-assisted portions and remaining work.

Practice a 3-minute demonstration: point to the divider; show live values and a DMM comparison; show a warming response; present a saved fault recording; explain one failure and its fix if you actually observed one. Save an offline hardware CSV and graph in case USB fails at the interview.

If the invitation is for this coming Wednesday, the working schedule is: first session identify parts/install software/demo; next session assemble/compile/electrical tests; final session faults/graphs/documentation/rehearsal. Confirm the invitation date yourself.

Official setup references

Espressif Arduino-ESP32 installation: https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html Espressif ADC API: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/adc.html pySerial port tools: https://pyserial.readthedocs.io/en/latest/tools.html Python Tkinter: https://docs.python.org/3/library/tkinter.html NIST individuals control charts: https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc322.htm